MPMC lock-free queue benchmarking with LTTng tracing

- LTTng tracing and conventional logging tools' comparison
- MPMC lock-free queues benchmarking and performance comparison

LTTng

Compare LTTng vs conventional logging libraries

- Key findings:
  - Data format and handling difference
  - LTTng overhead: ~30ns per event when active, 0ns when disabled
  - spdlog async: ~100-300ns per event
  - LTTng uses lock-free ring buffers, no blocking
  - Conventional logs cause contention under high load

LTTng

Feature	LTTng	spdlog	syslog
Blocking	No	Yes (can be)	Yes
Kernel tracing	Yes	No	No
Binary Output	Yes (CFT)	Text	Text
Use Case	Performance analysis	Application logging	System logging

LTTng

- **Conclusion**
  - LTTng is superior for performance tracing and debugging concurrency
  - Conventional logging is better for business logic and error reporting
  - Use LTTng when you need nanosecond precision and kernel correlation
  - Use spdlog when you need human-readable logs for operations

MPMC lock-free Queues

Four queues implemented:

1) MutexQueue - Baseline blocking queue (std::mutex + std::queue)
2) RingBufferQueue - Lock-free bounded queue (CAS on positions)
3) TwoLockHazardQueue - Michael-Scott unbounded queue with hazard pointers (after noticing ABA problem)

All share same API: enqueue(), dequeue(), size(), capacity()

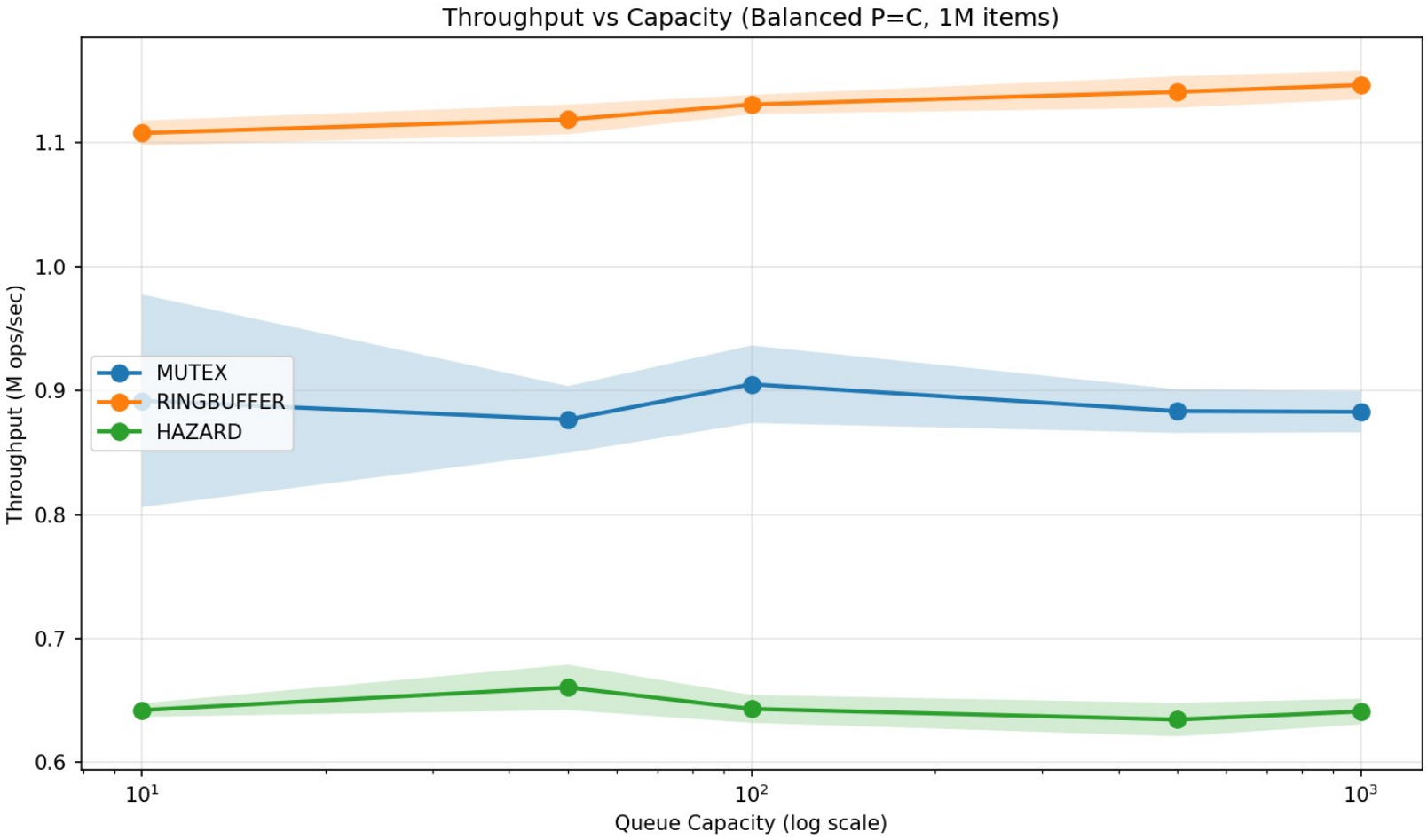
MPMC lock-free Queues

- Hardware: Ryzen 8845HS laptop (8 cores / 16 threads)
  power-save mode @1400Mhz
- Fixed configuration: 8 producers, 8 consumers, 1M items each
- Queue capacities tested: 10, 50, 100, 500, 1000
- Unbalanced tests: 14/2, 2/14, 15/1, 1/15
- Metrics: Throughput (M ops/sec), P99 latency (µs), failures
- LTTng tracing for latency measurement

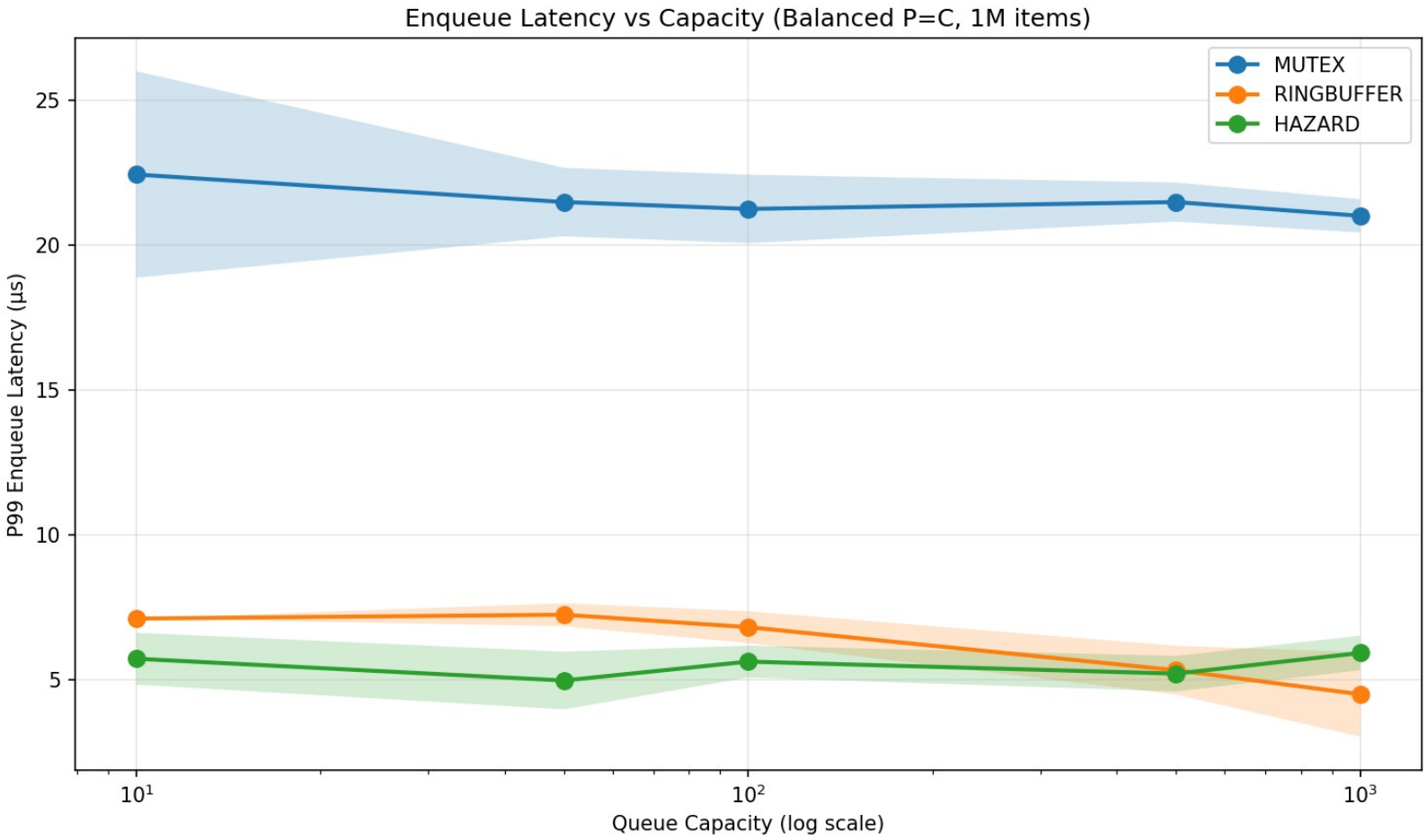
MPMC lock-free Queues

- LTTng successfully captures events with ~30ns overhead
- Major challenge: Matching trace events to specific benchmark runs
- Solution: Benchmark writes its PID to file, analysis uses PID to filter
- Without this, trace analysis is impossible

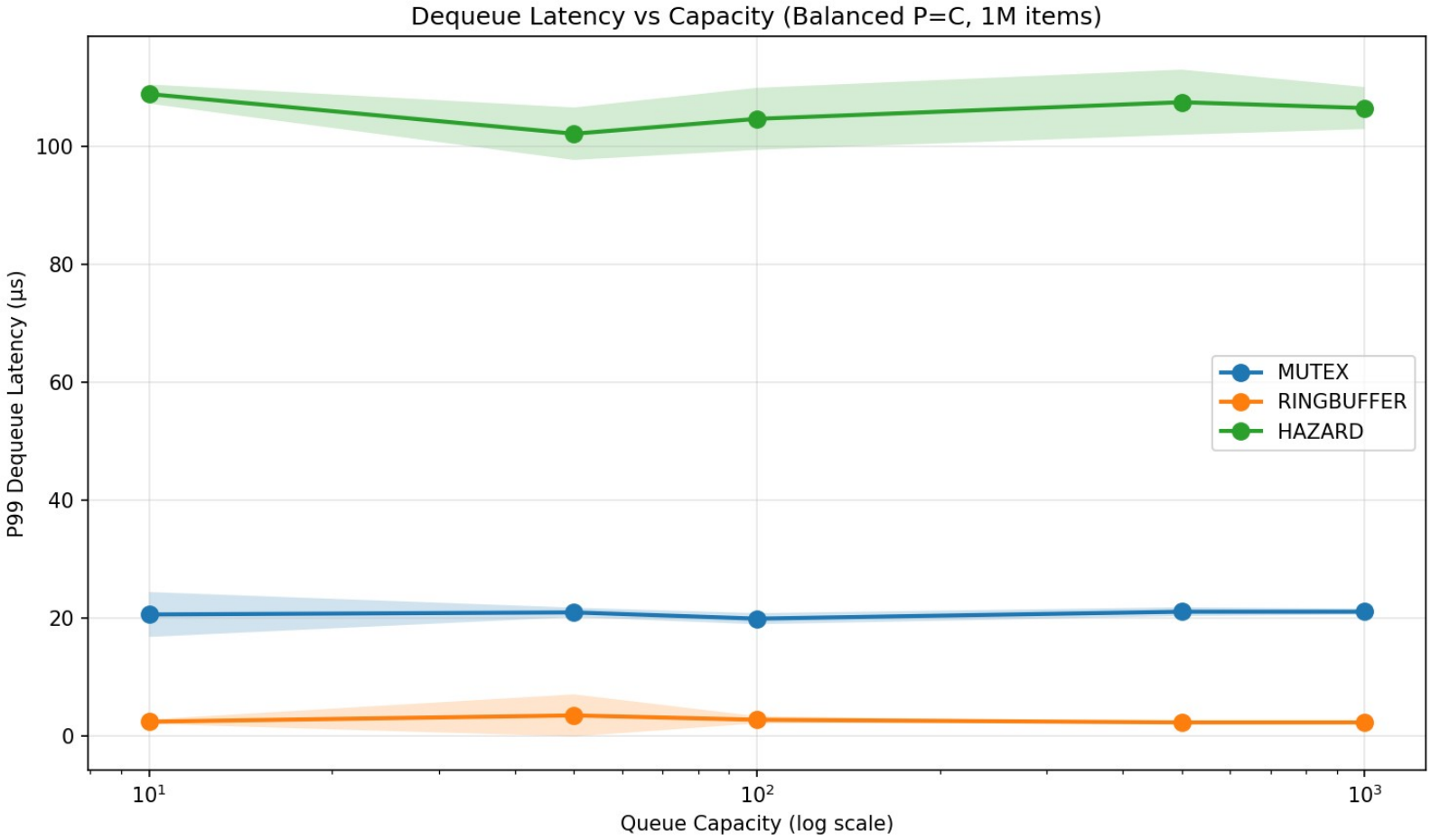
MPMC lock-free Queues



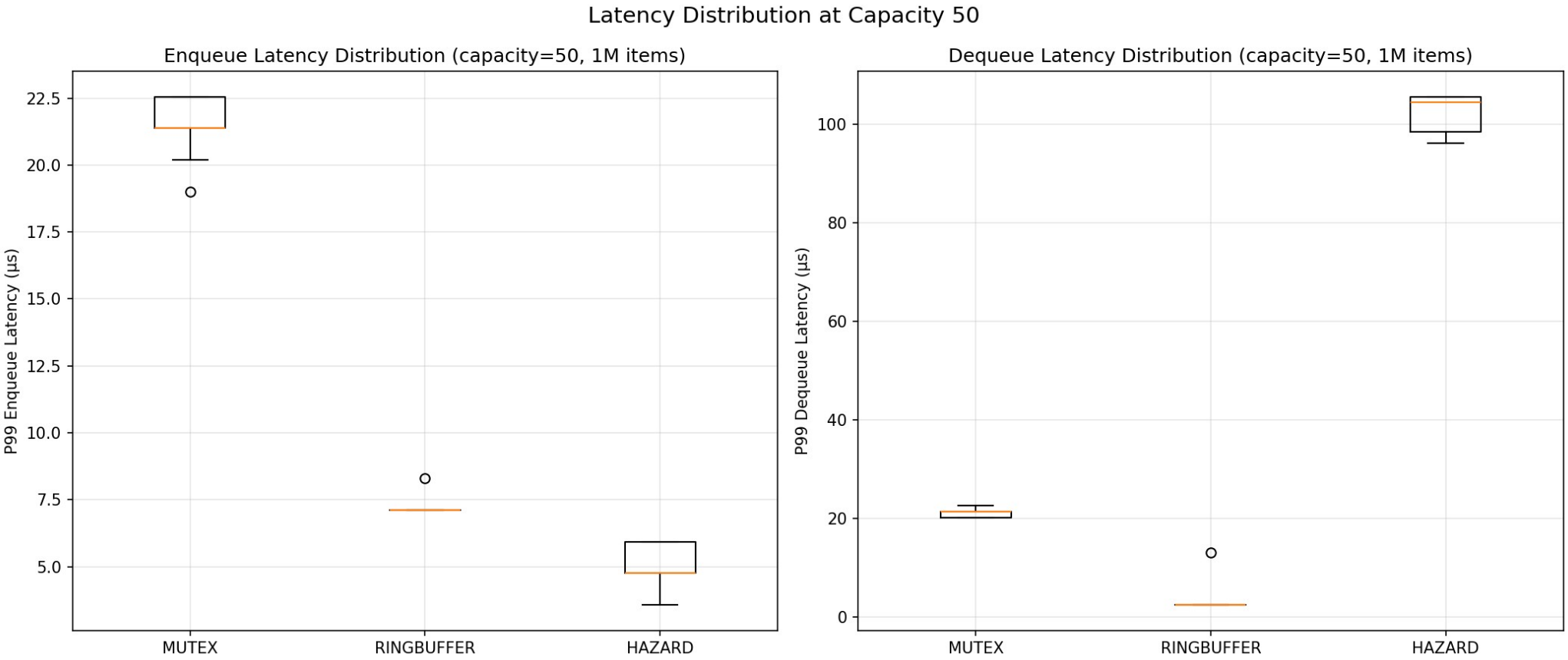
MPMC lock-free Queues



MPMC lock-free Queues



MPMC lock-free Queues



MPMC lock-free Queues

Hazard pointers solve the ABA problem correctly

- But they add significant overhead:
  - Each dequeue requires registering/unregistering hazard pointers
  - Periodic scans of retired nodes (O(N))
  - Cache contention on global hazard pointer array
- Result: Dequeue latency 103µs vs 2.4µs for ringbuffer
- Recommendation: Use bounded queues unless **unbounded** is absolutely required

MPMC lock-free Queues

- Conclusion
  - Ringbuffer (low latency, good throughput)
  - Mutex (good enough for many workloads)
  - Hazard pointers unless unbounded required
  - Lock-free is about latency, not throughput

Overall Conclusions

- PID tracking is critical for LTTng analysis
- Mutex can be competitive regarding throughput
- Lock-free wins on latency, not necessarily throughput
- Hazard pointers are correct but expensive
- Bounded queues are simpler and faster than unbounded
- LTTng traces require post-processing (babeltrace, Python)

Thank You for your attention!

Q&A